

Classes, Objects, and Methods

Introduction

- Java is an object-oriented programming (OOP) language. The fundamental building block of Java programs is the **class**. A class is a blueprint or template for creating objects. Objects are instances of classes, and they encapsulate data (variables) and behavior (methods).

Defining a Class

A class is defined using the class keyword, followed by the class name and a pair of curly braces {} enclosing the class body.

```
class MyClass {  
    // Class members (variables and methods) go  
    here  
}
```

Example:

```
class Dog {
```

```
// Class members will be defined here
```

```
}
```

Adding Variables (Data Members)

- Variables declared within a class are called **variables** or **data members**. They represent the state of an object. Each object of the class has its own copy of these variables.

- **EXAMPLE**

```
class Dog {  
    String breed;  
    String name;  
    int age;  
}
```

Adding Methods (Member Functions)

Methods define the behavior of an object. They are functions that operate on the object's data.

```
class Dog {  
    String breed;  
    String name;  
    int age;  
  
    void bark() {  
        System.out.println("Woof!");  
    }  
  
    void displayInfo() {  
        System.out.println("Breed: " + breed);  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

Here, `bark()` and `displayInfo()` are methods of the `Dog` class.

Creating Objects

An object is an instance of a class. To create an object, you use the new keyword followed by the class name and parentheses ().

Dog myDog = new Dog();

- This creates an object named myDog of the Dog class.

Accessing Class Members

- You can access the members (variables and methods) of an object using the dot operator ..

```
Dog myDog = new Dog();  
myDog.breed = "Golden Retriever";  
myDog.name = "Buddy";  
myDog.age = 3;
```

```
myDog.bark(); // Output: Woof!  
myDog.displayInfo();  
// Output:  
// Breed: Golden Retriever  
// Name: Buddy  
// Age: 3
```

Constructors

A constructor is a special method that is automatically called when an object of the class is created. It is used to initialize the object's state. A constructor has the same name as the class and does not have a return type.

```
class Dog {  
    String breed;  
    String name;  
    int age;  
  
    // Constructor  
    Dog(String breed, String name, int age) {  
        this.breed = breed;  
        this.name = name;  
        this.age = age;  
    }  
  
    void bark() {  
        System.out.println("Woof!");  
    }  
  
    void displayInfo() {  
        System.out.println("Breed: " + breed);  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog("Golden Retriever", "Buddy", 3);  
        myDog.displayInfo();  
    }  
}
```

Methods Overloading

Method overloading allows you to define multiple methods in the same class with the same name but different parameters (different number of parameters, different types of parameters, or different order of parameters). The compiler determines which method to call based on the arguments passed during the method call.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(2, 3));           // Output: 5  
        System.out.println(calc.add(2.5, 3.5));       // Output: 6.0  
        System.out.println(calc.add(2, 3, 4));        // Output: 9  
    }  
}
```

Static Members

- Static members (variables and methods) belong to the class itself rather than to any specific object of the class. They are shared by all objects of the class. You access static members using the class name instead of an object.

```
class Counter {  
    static int count = 0; // Static variable  
  
    Counter() {  
        count++;  
    }  
  
    static int getCount() { // Static method  
        return count;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Counter c1 = new Counter();  
        Counter c2 = new Counter();  
        Counter c3 = new Counter();  
  
        System.out.println(Counter.getCount()); // Output: 3  
    }  
}
```

Nesting of Methods

Nesting of methods refers to calling one method from within another method of the same class. This allows you to break down complex tasks into smaller, more manageable functions.

```
class MyClass {  
    void method1() {  
        System.out.println("Method 1 called");  
        method2(); // Calling method2 from method1  
    }  
  
    void method2() {  
        System.out.println("Method 2 called");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj.method1();  
        // Output:  
        // Method 1 called  
        // Method 2 called  
    }  
}
```

Extending a Class

Inheritance is a mechanism in which one class acquires the properties (fields and methods) of another class. The class that inherits is called the subclass or derived class, and the class from which it inherits is called the super class or base class. Inheritance promotes code reusability and establishes an "is-a" relationship between classes.

- In Java, we use the extends keyword to indicate that a class is inheriting from another class.

```
class Animal {
    String name;

    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.name = "Buddy"; // Inherited from Animal
        myDog.eat();           // Inherited from Animal
        myDog.bark();          // Defined in Dog
    }
}
```

Overriding Methods

Method overriding allows a subclass to provide a specific implementation for a method that is already defined in its super class. The method in the subclass must have the same signature (name, return type, and parameters) as the method in the super class.

```
class Animal {  
    void makeSound() {  
        System.out.println("Generic animal sound");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Meow");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal myAnimal = new Animal();  
        Cat myCat = new Cat();  
  
        myAnimal.makeSound(); // Output: Generic animal sound  
        myCat.makeSound();    // Output: Meow  
    }  
}
```

Final Variables and Methods

- The final keyword in Java is a non-access modifier that can be applied to variables, methods, and classes. Its meaning varies depending on the context.

Final Variables

- When a variable is declared as final, its value cannot be changed after it has been initialized. This makes it a constant. final variables must be initialized either at the time of declaration or within the constructor of the class.

```
public class Example {  
    final int x = 10; // Initialized at declaration  
  
    final int y;  
  
    public Example(int value) {  
        y = value; // Initialized in the constructor  
    }  
  
    public void modify() {  
        // x = 20; // Compilation error: cannot assign a value to final variable  
x  
        // y = 30; // Compilation error: cannot assign a value to final variable  
y  
    }  
}
```

Finalizer Methods

Finalizer methods, denoted by `finalize()`, are special methods in Java that are called by the garbage collector (GC) just before an object is garbage collected. They provide a last chance for an object to perform cleanup activities, such as releasing resources that are not automatically managed by the GC.

Syntax

The finalize() method is defined within a class as follows:

```
protected void finalize() throws Throwable {  
    // Cleanup code here  
    super.finalize();  
    // Call superclass's finalize method  
}
```

Explanation:

- `protected`: The `finalize()` method is typically declared as `protected` to allow subclasses to override it.
- `void`: It does not return any value.
- `throws Throwable`: It can throw any exception.
- `super.finalize()`: It's important to call the `finalize()` method of the super class to ensure that any cleanup activities defined in the super class are also executed.

Abstract Methods and Classes

Abstract Methods

An abstract method is a method declared without an implementation. It is defined using the abstract keyword.

Syntax:

Abstract returnType methodName(parameterList);

Example:

```
abstract class Shape {  
    public abstract double getArea();  
}
```

Abstract Classes

- An abstract class is a class that cannot be instantiated directly. It may contain abstract methods, concrete methods, or both. If a class contains at least one abstract method, it must be declared as abstract.
- **Syntax:**

```
abstract class ClassName {  
    // Fields and methods  
}
```

```
abstract class Shape {  
    private String color;  
  
    public Shape(String color) {  
        this.color = color;  
    }  
  
    public String getColor() {  
        return color;  
    }  
  
    public abstract double getArea();  
  
    public void displayColor() {  
        System.out.println("Color: " + color);  
    }  
}
```

Key Points:

- Abstract classes can have constructors, but they can only be called by subclasses.
- A subclass of an abstract class must either implement all the abstract methods of its superclass or declare itself as abstract.

Visibility Control (Access Modifiers)

- Visibility control, also known as access control, determines the accessibility of classes, methods, and variables from different parts of the program. Java provides four access modifiers:
 1. `private`: Accessible only within the declaring class.
 2. default (**`package-private`**): Accessible within the same package. No explicit modifier is used.
 3. `protected`: Accessible within the same package and by subclasses in other packages.
 4. `public`: Accessible from anywhere.

- Modifier | Class | Package | Subclass | World

private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

```
package com.example;

public class MyClass {
    private int privateVar; // Accessible only within MyClass
    int defaultVar;         // Accessible within the com.example package
    protected int protectedVar; // Accessible within the com.example package and
    by subclasses
    public int publicVar;    // Accessible from anywhere

    public void myMethod() {
        privateVar = 10; // OK
        defaultVar = 20; // OK
        protectedVar = 30; // OK
        publicVar = 40; // OK
    }
}

package com.example.another;

import com.example.MyClass;

class AnotherClass extends MyClass {
    public void anotherMethod() {
        //privateVar = 10; // Error: private
        defaultVar = 20;    // Error: different package
        protectedVar = 30; // OK: subclass
    }
}
```

```
        publicVar = 40; // OK
    }
}

package com.example.another;s

import com.example.MyClass;

class AnotherClass extends MyClass {
    public void anotherMethod() {
        //privateVar = 10; // Error: private
        defaultVar = 20;    // Error: different package
        protectedVar = 30; // OK: subclass
        publicVar = 40;     // OK
    }
}

package com.test;

import com.example.MyClass;

public class TestClass {
    public static void main(String[] args) {
```

```
MyClass obj = new MyClass();  
    //obj.privateVar = 10; // Error: private  
    //obj.defaultVar = 20; // Error: different package  
    //obj.protectedVar = 30; // Error: different package, not a subclass  
    obj.publicVar = 40; // OK  
}  
}
```

Arrays

- Arrays are fundamental data structures in Java used to store collections of elements of the same data type. They provide a way to organize and access multiple values using a single variable name.

- **One-Dimensional Arrays**

A one-dimensional array is a linear sequence of elements.

- **Creating an Array:**

There are two primary ways to create an array in Java:

1. Declaration and Initialization:

```
```java
```

```
int[] numbers = new int[5]; // Declares an integer array named 'numbers' with a
size of 5.
```

```
```
```

This creates an array capable of holding 5 integer values. The `new` keyword allocates memory for the array. The elements are initialized to their default values (0 for integers, `false` for booleans, `null` for objects).

2. Declaration, Initialization, and Assignment:

```
```java
```

```
int[] numbers = {1, 2, 3, 4, 5}; // Declares, initializes, and assigns values to
the array.
```

```
```
```

This method directly assigns values to the array elements during creation. The size of the array is implicitly determined by the number of elements provided.

Accessing Array Elements:

Array elements are accessed using their index, which starts from 0.

```
int[] numbers = {10, 20, 30, 40, 50};
```

```
System.out.println(numbers[0]);
```

```
// Output: 10 (accessing the first element)
```

```
System.out.println(numbers[3]);
```

```
// Output: 40 (accessing the fourth element)
```

Iterating Through an Array:

- Arrays can be iterated through using loops, such as for loops or enhanced for loops (also known as "for-each" loops).

```
int[] numbers = {1, 2, 3, 4, 5};
```

```
// Using a for loop
```

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

```
// Using an enhanced for loop
```

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

Two-Dimensional Arrays

- A two-dimensional array is an array of arrays, forming a grid or matrix-like structure.
- **Creating a Two-Dimensional Array:**

```
int[][] matrix = new int[3][4];
```

```
// Creates a 3x4 integer matrix.
```

Initializing a Two-Dimensional Array:

- `int[][] matrix = { {1, 2, 3, 4},
 {5, 6, 7, 8},
 {9, 10, 11, 12} };`

Accessing Elements:

Elements in a two-dimensional array are accessed using two indices: the row index and the column index.

```
System.out.println(matrix[0][0]); // Output: 1 (accessing the element at row 0,  
column 0)  
System.out.println(matrix[2][1]); // Output: 10 (accessing the element at row 2,  
column 1)
```

Iterating Through a Two-Dimensional Array:

Nested loops are typically used to iterate through two-dimensional arrays.

```
for (int i = 0; i < matrix.length; i++) {  
    for (int j = 0; j < matrix[i].length; j++) {  
        System.out.print(matrix[i][j] + " ");  
    }  
    System.out.println(); // Move to the next row  
}
```

Strings

- Strings in Java are immutable sequences of characters. The String class provides numerous methods for manipulating strings.
- **Creating Strings:**

```
String str1 = "Hello";
```

```
// String literal
```

```
String str2 = new String("World");
```

```
// Using the String constructor
```

String Methods:

- `length()` : Returns the length of the string.
- `charAt(int index)` : Returns the character at the specified index.
- `substring(int beginIndex, int endIndex)` : Returns a substring from `beginIndex` [inclusive] to `endIndex` [exclusive].
- `equals(String anotherString)` : Compares two strings for equality [case-sensitive].
- `equalsIgnoreCase(String anotherString)` : Compares two strings for equality [case-insensitive].
- `concat(String str)` : Concatenates the specified string to the end of this string.
- `toUpperCase()` : Converts the string to uppercase.
- `toLowerCase()` : Converts the string to lowercase.
- `trim()` : Removes leading and trailing whitespace.
- `replace(char oldChar, char newChar)` : Replaces all occurrences of `oldChar` with `newChar` .
- `indexOf(String str)` : Returns the index of the first occurrence of the specified substring.
- `lastIndexOf(String str)` : Returns the index of the last occurrence of the specified substring.
- `split(String regex)` : Splits the string into an array of substrings based on the given regular expression.

Vectors

- Vectors are dynamic arrays that can grow or shrink in size as needed. They are part of the `java.util` package. `Vector` is a legacy class, and `ArrayList` is generally preferred for new code due to its better performance. However, `Vector` is synchronized, making it thread-safe, while `ArrayList` is not.

Creating a Vector:

```
import java.util.Vector;  
  
Vector<Integer> numbers = new Vector<>(); // Creates an empty vector of  
integers.
```

Adding Elements:

```
numbers.add(10);  
numbers.add(20);  
numbers.add(30);
```

Accessing Elements:

```
System.out.println(numbers.get(0)); // Output: 10
```

Wrapper Classes

Wrapper classes are classes that encapsulate primitive datatypes. They provide a way to treat primitive types as objects.

- **Wrapper Classes for Primitive Types:**

- ✓ Integer for int
- ✓ Double for double
- ✓ Float for float
- ✓ Long for long
- ✓ Short for short
- ✓ Byte for byte
- ✓ Character for char
- ✓ Boolean for boolean

Creating Wrapper Objects

```
Integer num1 = new Integer(10);
```

```
// Creating an Integer object
```

```
Integer num2 = Integer.valueOf(20);
```

```
// Using the valueOf() method
```

- **Autoboxing and Unboxing:**

Java provides autoboxing and unboxing, which automatically convert between primitive types and their corresponding wrapper classes.

- **Autoboxing:** Automatic conversion of a primitive type to its corresponding wrapper class.

```
```java
```

```
Integer num = 10; // Autoboxing: int 10 is automatically converted to Integer
object.
```

```
```
```

- **Unboxing:** Automatic conversion of a wrapper class object to its corresponding primitive type.

```
```java
```

```
int number = num; // Unboxing: Integer object 'num' is automatically converted
to int.
```

```
```
```